

Program 2: Singly Linked List: Deletion and Display

Aim

To write a C program that deletes nodes from a singly linked list by value (first occurrence, all occurrences, or the whole list) and displays the list.

Theory

A linked list node holds a **data** field and a **next** pointer; **head** points to the first node and the last node points to NULL.

To delete a node, keep two pointers: **cur** (node being examined) and **prev** (node before it). Set `prev->next = cur->next` and free cur. If cur is the first node (prev is NULL), set `head = cur->next`.

Delete all occurrences of a key removes leading matches at the head first, then scans the rest, unlinking every matching node.

Delete entire list frees every node one by one and sets head to NULL, preventing memory leaks. Time complexity of all these operations is $O(n)$.

Algorithm

1. Search the list using cur and prev until cur->data equals the key.
2. If cur is NULL, report that the value is not found.
3. If prev is NULL, set head = cur->next; otherwise set prev->next = cur->next.
4. free(cur).
5. For all occurrences: remove matching head nodes, then loop over the list unlinking every matching next node.
6. For deleting the whole list: while head is not NULL, save head, move head forward, free the saved node.
7. Display the list after each operation.

C Source Code

```
/*
 * Program : Singly Linked List - Deletion (by value) and Display
 *
 * THEORY:
 * - To delete a node, the pointer of the PREVIOUS node is changed to skip
 *   the node, and the removed node is released with free().
 * - Deleting the head node is a special case: head moves to head->next.
 * - Delete by value : search for the first node holding the key and unlink it.
 * - Delete all occurrences : keep scanning the list after every removal.
 * - Delete entire list : free every node one by one and set head = NULL.
 * - Time complexity:  $O(n)$  because the list may have to be scanned.
 */
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};
struct Node *head = NULL;

void insertEnd(int val) {
```

/* used to build the list */

```

    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->data = val;
    n->next = NULL;
    if (head == NULL) head = n;
    else {
        struct Node *t = head;
        while (t->next != NULL) t = t->next;
        t->next = n;
    }
}

void deleteByValue(int key) {
    struct Node *cur = head, *prev = NULL;
    while (cur != NULL && cur->data != key) { /* find the node */
        prev = cur;
        cur = cur->next;
    }
    if (cur == NULL) { printf("Value %d not found.\n", key); return; }
    if (prev == NULL) head = cur->next; /* deleting the first node */
    else prev->next = cur->next;
    free(cur);
    printf("Deleted first occurrence of %d.\n", key);
}

void deleteAllOccurrences(int key) {
    int count = 0;
    while (head != NULL && head->data == key) { /* leading matches */
        struct Node *t = head; head = head->next; free(t); count++;
    }
    struct Node *cur = head;
    while (cur != NULL && cur->next != NULL) {
        if (cur->next->data == key) {
            struct Node *del = cur->next;
            cur->next = del->next;
            free(del);
            count++;
        } else
            cur = cur->next;
    }
    if (count) printf("Deleted %d node(s) with value %d.\n", count, key);
    else printf("Value %d not found.\n", key);
}

void deleteList() {
    while (head != NULL) {
        struct Node *t = head; head = head->next; free(t);
    }
    printf("Entire list deleted.\n");
}

void display() {
    if (head == NULL) { printf("List is empty.\n"); return; }
    printf("List: ");
    for (struct Node *t = head; t != NULL; t = t->next)
        printf("%d -> ", t->data);
    printf("NULL\n");
}

int main() {
    int choice, val;
    do {
        printf("\n1.Add node at end 2.Delete by value 3.Delete all occurrences\n"
            "4.Delete entire list 5.Display 6.Exit\nEnter choice: ");
        scanf("%d", &choice);
        switch (choice) {
            case 1: printf("Enter value: "); scanf("%d", &val); insertEnd(val); break;
            case 2: printf("Enter value to delete: "); scanf("%d", &val); deleteByValue(val); break;
            case 3: printf("Enter value to delete: "); scanf("%d", &val); deleteAllOccurrences(val); break;
            case 4: deleteList(); break;
            case 5: display(); break;
            case 6: printf("Exiting...\n"); break;
        }
    } while (choice != 6);
}

```

```

        default: printf("Invalid choice!\n");
    }
} while (choice != 6);
deleteList();
return 0;
}

```

Sample Input

Build list 10, 20, 10, 30, 10 -> Display -> Delete first 20 -> Display -> Delete all 10s -> Display -> Delete entire list -> Display -> Exit.

Sample Output (program run with the input above)

```

1.Add node at end  2.Delete by value  3.Delete all occurrences
4.Delete entire list  5.Display  6.Exit
Enter choice: Enter value:
1.Add node at end  2.Delete by value  3.Delete all occurrences
4.Delete entire list  5.Display  6.Exit
Enter choice: Enter value:
1.Add node at end  2.Delete by value  3.Delete all occurrences
4.Delete entire list  5.Display  6.Exit
Enter choice: Enter value:
1.Add node at end  2.Delete by value  3.Delete all occurrences
4.Delete entire list  5.Display  6.Exit
Enter choice: Enter value:
1.Add node at end  2.Delete by value  3.Delete all occurrences
4.Delete entire list  5.Display  6.Exit
Enter choice: List: 10 -> 20 -> 10 -> 30 -> 10 -> NULL

1.Add node at end  2.Delete by value  3.Delete all occurrences
4.Delete entire list  5.Display  6.Exit
Enter choice: Enter value to delete: Deleted first occurrence of 20.

1.Add node at end  2.Delete by value  3.Delete all occurrences
4.Delete entire list  5.Display  6.Exit
Enter choice: List: 10 -> 10 -> 30 -> 10 -> NULL

1.Add node at end  2.Delete by value  3.Delete all occurrences
4.Delete entire list  5.Display  6.Exit
Enter choice: Enter value to delete: Deleted 3 node(s) with value 10.

1.Add node at end  2.Delete by value  3.Delete all occurrences
4.Delete entire list  5.Display  6.Exit
Enter choice: List: 30 -> NULL

1.Add node at end  2.Delete by value  3.Delete all occurrences
4.Delete entire list  5.Display  6.Exit
Enter choice: Entire list deleted.

1.Add node at end  2.Delete by value  3.Delete all occurrences
4.Delete entire list  5.Display  6.Exit
Enter choice: List is empty.

1.Add node at end  2.Delete by value  3.Delete all occurrences
4.Delete entire list  5.Display  6.Exit
Enter choice: Exiting...
Entire list deleted.

```